Terraform FAQ

1. What is Terraform and why is it used?

Answer:

- Terraform is an **Infrastructure as Code (IaC)** tool to provision, manage, and version cloud resources.
- It allows automation, reproducibility, and consistency across environments.
- Supports multiple cloud providers (AWS, Azure, GCP) and on-prem infrastructure.

---

2. What is the difference between Terraform and configuration management tools like Ansible?

Answer:

- Terraform is **declarative**, managing infrastructure lifecycle.
- Ansible is primarily **imperative**, focusing on configuration and software deployment.
- Terraform creates resources, Ansible configures them after creation.

---

3. What is a Terraform provider?

Answer:

- A provider is a plugin that interacts with a cloud or service API.
- Examples: AWS, Azure, GCP, Kubernetes.
- Terraform uses providers to create and manage resources.

---

4. What is a Terraform resource?

Answer:

- A resource represents a component of infrastructure, e.g., an EC2 instance, S3 bucket, or VPC.
- Terraform manages the lifecycle of resources: create, update, and delete.
- Resources are the main building blocks in Terraform configurations.

---

5. What is Terraform state and why is it important?

Answer:

- Terraform state tracks **current infrastructure status**.
- Used to map Terraform resources to real-world infrastructure.
- Enables planning, detecting drift, and incremental updates.

---

6. What is a Terraform module?

Answer:

- A module is a **reusable, self-contained configuration**.
- Promotes code modularity and reuse across projects or environments.
- Modules can have inputs (variables) and outputs.

---

7. What is the difference between local and remote backends in Terraform?

Answer:

- Local backend stores state files on your machine.
- Remote backend stores state remotely (S3, Terraform Cloud, GCS) with locking and team collaboration.
- Remote backends prevent conflicts and support multi-user environments.

---

8. What is a Terraform plan?

Answer:

- `terraform plan` previews changes Terraform will make.
- Shows resources to be created, updated, or destroyed.
- Helps prevent accidental changes to infrastructure.

---

9. What is `terraform apply`?

Answer:

- Applies changes to reach the desired infrastructure state.
- Reads configuration and state to create, update, or delete resources.
- Terraform ensures resources match the declared configuration.

---

10. What is `terraform destroy`?

Answer:

- Removes all resources defined in Terraform configuration.
- Used for clean-up or decommissioning environments.
- Can also target specific resources if needed.

---

11. What is a variable in Terraform?

Answer:

- Variables make configurations **dynamic and reusable**.
- They can be passed via CLI, `.tfvars` files, or environment variables.
- Helps manage configurations for multiple environments.

---

12. What is an output in Terraform?

Answer:

- Outputs expose information about resources after `terraform apply`.
- Useful for sharing resource IDs, IPs, or endpoints with other modules or scripts.
- Enables modular and dynamic infrastructure management.

---

13. What is the difference between `count` and `for_each` in Terraform?

Answer:

- `count` → creates multiple identical resources using a numeric value.
- `for_each` → creates multiple resources from a map or set with unique properties.
- `for_each` provides more flexibility for dynamic and parameterized resources.

---

14. How does Terraform handle dependencies between resources?

Answer:

- Terraform automatically detects dependencies via resource references.
- `depends_on` can be used explicitly if dependencies aren't obvious.
- Ensures resources are created in the correct order to avoid errors.

---

15. How can you manage sensitive data in Terraform?

Answer:

- Use sensitive variables and secret management systems (Vault, Secrets Manager, Parameter Store).
- Mark variables as sensitive to prevent logging in outputs or state files.
- Ensures credentials and secrets are securely handled in Terraform configurations.

16. Scenario: Use Terraform modules effectively. How?

Answer:

- Modules encapsulate reusable code for infrastructure components.
- Allows sharing and reusing configurations across environments.
- Outputs from modules can be referenced by other modules for dependency management.

---

17. Scenario: Use Terraform workspaces for multiple environments. How?

Answer:

- Workspaces allow separate states for dev, staging, and prod.
- Same code can be reused across environments without conflict.
- Useful for isolating resources per environment.

---

18. Scenario: Manage Terraform state securely. How?

Answer:

- Use remote backends like S3, Terraform Cloud, or GCS with locking.
- Prevents corruption and allows team collaboration.
- Local state should be avoided for shared projects.

---

19. Scenario: Handle Terraform drift in production. How?

Answer:

- Drift occurs when manual changes are made outside Terraform.
- Use `terraform plan` regularly to detect drift.
- Update configurations to reflect manual changes or revert manual changes via Terraform.

---

20. Scenario: Provision resources dynamically based on variable inputs. How?

Answer:

- Use `for_each` or `count` with variables to create resources dynamically.
- Each resource can have unique properties without duplicating code.
- Makes infrastructure scalable and configurable.

---

21. Scenario: Securely pass sensitive information to Terraform. How?

Answer:

- Use environment variables, secret managers, or parameter stores.
- Mark variables as sensitive in Terraform to avoid logging.
- Ensures credentials or passwords are not exposed in logs or state files.

---

22. Scenario: Handle multiple providers in a single Terraform project. How?

Answer:

- Use **provider aliases** to configure multiple providers.
- Assign resources to the correct provider.
- Supports multi-cloud or multi-account setups in one project.

---

23. Scenario: Implement resource lifecycle management. How?

Answer:

- Use lifecycle rules like `prevent_destroy` and `ignore_changes`.
- Prevents accidental deletion or recreation of critical resources.
- Maintains stability in production environments.

---

24. Scenario: Use Terraform outputs to share information between modules. How?

Answer:

- Outputs allow exposing resource attributes from one module.
- Other modules or projects can reference these outputs.
- Promotes modularity and dependency management.

---

25. Scenario: Upgrade Terraform versions safely. How?

Answer:

- Test upgrades in a non-production environment first.
- Use `terraform plan` to preview changes.
- Ensure modules and providers are compatible before applying changes.

---

26. Scenario: Handle resource dependencies explicitly. How?

Answer:

- Terraform usually detects dependencies automatically via references.
- Use `depends_on` only when Terraform cannot infer dependencies.
- Ensures resources are created, updated, or deleted in the correct order.

---

27. Scenario: Import existing resources into Terraform. How?

Answer:

- Use `terraform import` to bring unmanaged resources under Terraform control.
- Write corresponding Terraform configuration to match imported resources.
- Ensures safe management without recreating existing infrastructure.

---

28. Scenario: Use dynamic blocks for flexible configurations. How?

Answer:

- Dynamic blocks allow creating nested or repeated resource configurations based on input data.
- Useful for security rules, network interfaces, or other repeated configurations.
- Reduces repetitive code and supports scalability.

---

29. Scenario: Use versioning for Terraform modules. Why is it important?

Answer:

- Versioning ensures stability and backward compatibility.
- Avoids breaking changes when modules are updated.
- Modules can be referenced with specific versions in Terraform projects.

---

30. Scenario: Automate resource cleanup in Terraform. How?

Answer:

- Use targeted destroy for specific resources.
- Use tags to identify temporary or test resources.
- Helps prevent unnecessary costs and resource sprawl.

---

31. Scenario: Provision a resource only if a certain condition is met. How?

Answer:

- Use conditional expressions or flags to control resource creation.
- Only creates the resource when the condition evaluates to true.
- Useful for environments with optional infrastructure components.

---

32. Scenario: Handle Terraform state conflicts in a team environment. How?

Answer:

- Use a **remote backend with state locking** (e.g., S3 + DynamoDB).
- Avoid multiple engineers applying changes simultaneously.
- Ensures consistency and prevents state corruption.

---

33. Scenario: Enforce all EC2 instances to use a specific AMI. How?

Answer:

- Use variables or data sources to standardize the AMI across all instances.
- Ensures consistency and compliance across environments.
- Prevents accidental use of unauthorized AMIs.

---

34. Scenario: Prevent unwanted recreation of resources. How?

Answer:

- Use lifecycle management to **ignore specific changes** or prevent destruction.
- Protects critical resources from accidental recreation or deletion.
- Helps maintain stable infrastructure.

---

35. Scenario: Create multiple similar resources with slight differences. How?

Answer:

- Use mapping structures and iteration (like `for_each`) to create resources dynamically.
- Each resource can have unique attributes without duplicating code.
- Ensures scalable and maintainable infrastructure.

---

36. Scenario: Migrate Terraform state from one backend to another. How?

Answer:

- Configure the new backend.
- Use `terraform init -migrate-state` to safely move the state.
- Ensures safe transition and collaboration without losing existing state.

---

37. Scenario: Provision cross-account resources securely. How?

Answer:

- Use multiple provider configurations with `assume_role` for the target account.
- Maintain separate state files or workspaces per account.
- Ensures resources are provisioned safely across accounts.

---

38. Scenario: Create dynamic nested resources, e.g., multiple security group rules. How?

Answer:

- Use dynamic blocks or iterations to generate multiple nested resources based on input data.
- Allows flexibility without duplicating code.
- Supports scalable and maintainable infrastructure.

---

39. Scenario: Automate cleanup of unused Terraform resources. How?

Answer:

- Use targeted destroy to remove specific resources.
- Tag temporary or test resources for easy identification.
- Helps prevent resource sprawl and unnecessary costs.

---

40. Scenario: Manage secrets securely in Terraform for CI/CD pipelines. How?

Answer:

- Use environment variables or secret management systems (Secrets Manager, Vault).
- Mark sensitive variables to prevent logging.
- Ensures secure handling of credentials and secrets.

---

41. Scenario: Scale an Auto Scaling Group using Terraform. How?

Answer:

- Define ASG parameters like min, max, and desired capacity.
- Use scaling policies and metrics to automate scaling.
- Terraform manages the infrastructure consistently with cloud scaling requirements.

---

42. Scenario: Prevent Terraform from overwriting manually configured DNS records. How?

Answer:

- Use lifecycle rules to **ignore specific changes** (like record values).
- Ensures manual updates are preserved while Terraform manages other attributes.
- Prevents accidental disruption of live DNS records.

---

43. Scenario: Version control Terraform modules for backward compatibility. How?

Answer:

- Use Git tags and semantic versioning.
- Reference specific module versions in Terraform to prevent breaking changes.
- Ensures stable, predictable infrastructure deployments.

---

44. Scenario: Manage Terraform configurations for multiple AWS accounts and regions. How?

Answer:

- Use provider aliases for accounts and regions.
- Use workspaces or separate state files per account/region.

- Maintains isolation and prevents accidental cross-environment changes.

---

45. Scenario: Run Terraform safely in a team with multiple developers. How?

Answer:

- Use remote backend with locking.
- Enforce code reviews and CI/CD validations (`fmt` and `validate`).
- Use separate workspaces or directories per environment.
- Avoid manual changes outside Terraform to prevent drift.

46. Difference between declarative and imperative approach in Terraform

Answer:

- Declarative: You define the desired state, and Terraform decides the steps to achieve it.
- Imperative: You define the exact steps to execute (like scripts).
- Terraform uses a declarative approach, making infrastructure predictable and idempotent.

---

47. Provision an EC2 instance and attach an existing security group. How?

Answer:

- Reference the security group resource in your EC2 configuration.
- Terraform automatically handles dependencies between the EC2 instance and the security group.

---

48. What are Terraform variables and why are they used?

Answer:

- Variables make configurations **dynamic and reusable**.
- They allow you to define inputs like instance types, regions, or names, instead of hardcoding values.
- Can be overridden via CLI, environment variables, or `.tfvars` files.

---

49. How do Terraform outputs work?

Answer:

- Outputs expose resource attributes after `apply`.
- They are used for referencing values across modules or sharing data with other systems or scripts.
- Example: Output an EC2 instance ID or VPC ID for downstream modules.

---

50. Create an S3 bucket only if it doesn't exist. How?

Answer:

- Use a **conditional flag** or `count` to control resource creation.
- Existing resources can be **imported** into Terraform to prevent recreation.
- Helps avoid conflicts with manually created resources.

---

51. Common Terraform commands

Answer:

- `terraform init` → Initialize Terraform.
- `terraform plan` → Preview changes.
- `terraform apply` → Apply changes.
- `terraform destroy` → Remove resources.
- `terraform fmt` → Format code.
- `terraform validate` → Check syntax.
- `terraform state` → Inspect or manage state.

---

52. Attach multiple IAM policies to a single IAM role

Answer:

- Use multiple policy attachments referencing the same IAM role.
- Can loop through policies using `for_each` for scalability.
- Avoids duplicating code for each policy attachment.

---

53. Purpose of `terraform fmt` and `terraform validate`

Answer:

- `terraform fmt` → Automatically formats code to HCL standards.

- `terraform validate` → Checks syntax and configuration correctness without applying changes.
- Both improve code quality and prevent errors.

---

54. Deploy a VPC with public and private subnets. How?

Answer:

- Create public subnets with Internet Gateway and private subnets with NAT Gateway.
- Use variables for CIDRs, AZs, and subnet count to make it reusable.
- Outputs provide subnet IDs for other modules like EC2 or RDS.

---

55. How to handle Terraform provider versioning?

Answer:

- Specify required provider versions in Terraform configuration.
- Prevents breaking changes when upgrading providers.
- Test upgrades in staging before production.

---

56. Update only resource tags without recreating it

Answer:

- Use lifecycle management to ignore changes to other attributes.
- Terraform updates only the desired attributes (tags) without destroying the resource.
- Prevents accidental downtime or disruption.

---

57. Difference between `count` and `for_each`

Answer:

- `count` → Creates multiple identical resources using a numeric value.
- `for_each` → Creates multiple resources from a map or set with unique attributes.
- `for_each` is more flexible for dynamic resource creation.

---

58. Deploy a Lambda function using S3 for code storage

Answer:

- Store Lambda code in S3 and reference it in Terraform.

- Environment variables or secrets can be configured without hardcoding.
- Ensures versioned and centralized deployment of Lambda functions.

---

59. Use outputs from one module in another

Answer:

- Modules can expose outputs like VPC IDs, subnet IDs, or security group IDs.
- Other modules reference these outputs for proper dependency management.
- Promotes modularity and reusability of Terraform code.

---

60. Destroy only a specific resource safely

Answer:

- Use targeted destroy to remove only the intended resource.
- Terraform ensures other resources remain untouched.
- Useful for cleanup or temporary resources in production or dev environments.

61. Scenario: You need to provision multiple AWS accounts using Terraform. How would you structure it?

Answer:

- Use **provider aliases** and `assume_role` for each account.
- Maintain **separate state files or workspaces** per account.
- Ensures isolation and prevents accidental cross-account changes.

---

62. Scenario: Deploy a multi-tier application with dependencies (VPC → subnets → EC2). How do you manage dependencies?

Answer:

- Use **resource references** for implicit dependencies.
- Use `depends_on` only if Terraform cannot infer dependencies.
- Ensures resources are created in the correct order.

---

63. Scenario: Provision a resource only if a specific environment variable exists. How?

Answer:

- Use **conditional expressions** or `count` with a boolean flag.
- Only provisions the resource when the condition is true.
- Helps avoid unnecessary resources in certain environments.

---

64. Scenario: Deploy Kubernetes clusters in multiple regions with Terraform. How do you manage it?

Answer:

- Use **provider aliases** per region.
- Use modules for cluster creation and outputs for kubeconfig.
- Keeps multi-region clusters organized and maintainable.

---

65. Scenario: Manage secrets for Lambda functions securely. How?

Answer:

- Use **Secrets Manager, SSM, or Vault**.
- Fetch secrets dynamically via Terraform data sources.
- Avoid hardcoding credentials; ensures security across environments.

---

66. Scenario: Upgrade a Terraform provider without breaking resources. How?

Answer:

- Pin provider versions in `required_providers`.
- Test in **staging workspace** first.
- Run `terraform plan` to see potential impacts before `apply`.

---

67. Scenario: Implement blue-green deployment for a web application. How?

Answer:

- Deploy two identical sets of infrastructure (blue & green).
- Switch traffic via **ALB rules or DNS**.
- Terraform manages infrastructure; traffic switching can be handled externally.

---

68. Scenario: Import existing infrastructure into Terraform safely. How?

Answer:

- Use `terraform import` for existing resources.
- Write matching Terraform configuration for imported resources.
- Ensures Terraform can manage pre-existing infrastructure without recreating it.

---

69. Scenario: Automate creation of security group rules based on input data. How?

Answer:

- Use **dynamic blocks** or `for_each` with a map.
- Allows scalable creation of multiple rules without duplicating code.
- Makes infrastructure flexible and maintainable.

---

70. Scenario: Manage multi-cloud infrastructure with Terraform. How?

Answer:

- Use **multiple providers** (AWS, Azure, GCP).
- Organize resources per provider.
- Use separate state files or workspaces to prevent conflicts.

---

71. Scenario: Rotate secrets used by Terraform-managed resources periodically. How?

Answer:

- Store secrets in **Vault or Secrets Manager** with rotation enabled.
- Fetch secrets dynamically in Terraform.
- Ensures resources always use updated secrets without manual updates.

---

72. Scenario: Prevent accidental deletion of production resources. How?

Answer:

- Use `lifecycle { prevent_destroy = true }`.
- Combine with **CI/CD approval workflows** for extra safety.
- Protects critical infrastructure from accidental deletion.

---

73. Scenario: Scale an Auto Scaling Group automatically based on metrics. How?

Answer:

- Use **CloudWatch alarms** and ASG scaling policies.
- Terraform manages both the ASG and the associated alarms.
- Ensures infrastructure scales automatically with demand.

---

74. Scenario: Migrate Terraform state from local to Terraform Cloud. How?

Answer:

- Configure **Terraform Cloud remote backend**.
- Run `terraform init -migrate-state` to move local state.
- Enables team collaboration, remote state storage, and locking.

---

75. Scenario: Enforce tagging and prevent public S3 buckets using policy-as-code. How?

Answer:

- Use **Sentinel or Open Policy Agent (OPA)**.
- Define policies for compliance, e.g., mandatory tags or blocking public access.
- Ensures organization-wide governance automatically.

76. What are the key best practices to follow when using Terraform in real-world projects?

Answer:

- **Use Version Control:** Store all Terraform code in Git for tracking changes and collaboration.
- **Use Remote State:** Keep state files in remote backends (S3, GCS, Terraform Cloud) with locking for team environments.
- **Separate Environments:** Use workspaces or separate state files for dev, staging, and production to prevent accidental changes.
- **Modularize Code:** Break infrastructure into reusable modules for consistency and maintainability.

- **Pin Versions:** Specify Terraform and provider versions to avoid breaking changes.
- **Use Variables and Avoid Hardcoding:** Make code dynamic and environment-agnostic.
- **Mark Sensitive Variables:** Protect secrets and credentials using sensitive flags and secret managers.
- **Manage Dependencies Carefully:** Use references or `depends_on` to ensure proper resource creation order.
- **Use Lifecycle Rules:** Protect critical resources with `prevent_destroy` and control updates with `ignore_changes`.
- **Use Outputs Properly:** Expose module outputs for referencing in other modules to reduce duplication.
- **Format and Validate Code:** Use `terraform fmt` and `terraform validate` to maintain code quality.
- **Plan Before Apply:** Always review `terraform plan` before applying changes.
- **Import Existing Resources:** Use `terraform import` to safely manage pre-existing infrastructure.
- **Tag Resources and Standardize Naming:** Helps with monitoring, auditing, and cost tracking.
- **Avoid Monolithic Configurations:** Break large projects into smaller, manageable modules.
- **Automate via CI/CD:** Integrate Terraform in pipelines for automated validation, plan, and apply.
- **Monitor Drift:** Regularly check for configuration drift and reconcile changes as needed.

77. Scenario:

You need to enforce **multi-team access control** for Terraform deployments. How do you handle it?
Answer:

- Use Terraform Cloud/Enterprise with RBAC.
- Split infra into separate workspaces/projects per team.
- Integrate with SSO/LDAP for role-based permissions.

---

78. Scenario:

A production deployment failed halfway, leaving some resources created and others not. How do you recover?
Answer:

- Run `terraform plan` again → Terraform reconciles desired vs actual.

- Roll forward (complete missing resources) or roll back (destroy incomplete infra).
- For critical workloads, use **manual state manipulation** (`terraform state rm/mv`).

---

79. Scenario:

How do you **track Terraform resource changes** for auditing?
Answer:

- Enable remote backend with versioned state (e.g., S3 with versioning).
- Store Terraform plans and apply logs in Git/CI system.
- Use audit integrations in Terraform Cloud/Enterprise.

---

80. Scenario:

You want **faster Terraform runs** in large projects. What optimizations would you apply?
Answer:

- Use `-target` for partial applies when safe.
- Break infra into smaller modules/states.
- Enable parallelism (`terraform apply -parallelism=N`).
- Cache providers/plugins locally in CI/CD.

---

81. Scenario:

Terraform keeps recreating a resource unnecessarily. How do you debug it?
Answer:

- Check `terraform plan` output for which attribute is changing.
- Use `ignore_changes` lifecycle rule if the change is managed externally.
- Verify provider schema (some fields may be marked “ForceNew”).

---

82. Scenario:

How do you enforce **cost control** using Terraform?
Answer:

- Implement tagging standards for cost allocation.
- Use policies (Sentinel/OPA) to restrict expensive resource types.
- Integrate cost estimation tools (Infracost, Terraform Cloud Cost Estimation).

---

83. Scenario:

You must manage **different Terraform versions** across multiple projects. How?
Answer:

- Pin versions in `required_version`.
- Use version managers (tfenv, asdf).
- Use Docker images or Terraform Cloud with locked versions.

---

84. Scenario:

How do you **test Terraform code** before production?
Answer:

- `terraform validate` & `terraform fmt` for syntax.
- `terraform plan` in CI for dry-runs.
- Use tools like Terratest or Checkov for unit/integration tests.
- Deploy in staging workspace first.

---

85. Scenario:

A team needs to provision infra in **10 AWS regions simultaneously**. How do you structure it?
Answer:

- Use provider aliases for each region.
- Iterate with `for_each` across regions.
- Keep separate state per region to avoid conflicts.

---

86. Scenario:

You need to ensure **all resources have mandatory tags**. How do you enforce this?
Answer:

- Use `default_tags` in provider config.
- Add Sentinel/OPA policies to block non-compliant resources.
- Validate via CI/CD pipelines before apply.

---

87. Scenario:

Terraform apply failed due to API rate limits. How do you handle it?
Answer:

- Use `-parallelism` to reduce API calls.
- Add retries/backoff with provider config.
- Split infra into multiple smaller applies.

---

88. Scenario:

Your Terraform state file is getting too large. How do you manage it?
Answer:

- Break infra into multiple states (network, compute, storage).
- Use modules with separate backends.
- Prune unused resources from state.

---

89. Scenario:

How do you implement **blue-green infrastructure deployment** using Terraform?
Answer:

- Duplicate infra stack (`blue` and `green`).
- Use `count` or `for_each` to control active environment.
- Switch traffic with DNS/ALB rules.

---

90. Scenario:

Terraform plan shows **destroy + recreate** for critical DB. How do you prevent downtime?
Answer:

- Investigate why → immutable attribute changed.
- Use `ignore_changes` if safe.
- Use provider-specific upgrade paths (e.g., RDS storage scaling).
- Apply in maintenance window with backups.

---

91. Scenario:

You need to provision **on-prem resources** with Terraform. How?
Answer:

- Use providers for VMware, OpenStack, vSphere.
- For custom systems, use `null_resource` with provisioners or external providers.
- Store hybrid state in remote backend.

---

92. Scenario:

How do you **secure Terraform state** that contains sensitive data?
Answer:

- Store in encrypted backend (S3 + SSE, GCS + CMEK, Terraform Cloud).
- Enable at-rest and in-transit encryption.
- Avoid committing state to Git.

---

93. Scenario:

You need to migrate **100+ existing resources** into Terraform. How do you plan it?
Answer:

- Use `terraform import` iteratively.
- Write config to match existing infra.
- Automate import with scripts (`terraformer`).

---

94. Scenario:

How do you run **Terraform safely in CI/CD pipelines**?
Answer:

- Split workflow: `terraform fmt` → `validate` → `plan` → `apply`.
- Require manual approval for production apply.
- Use service accounts with least privilege IAM.

---

95. Scenario:

Your team wants **multi-cloud DR (AWS + Azure)** with Terraform. How would you design it?
Answer:

- Use multiple providers in same config.
- Deploy identical infra in both clouds.
- Keep state per provider.
- Use DNS/global load balancing for failover.

---

96. Scenario:

Terraform apply is **very slow** due to thousands of resources. How do you optimize?
Answer:

- Break infra into layered stacks.
- Increase parallelism.
- Use modules for repeated patterns.
- Run selective applies with `-target`.

---

97. Scenario:

You need to handle **feature flags for infrastructure**. How?
Answer:

- Use conditionals or boolean variables to toggle infra components.
- Wrap optional resources in `count` or `for_each`.
- Helps test experimental infra safely.

---

98. Scenario:

How do you manage **Terraform in a GitOps workflow**?
Answer:

- Store all configs in Git.
- Trigger CI/CD on PR merges.
- Use PRs for `terraform plan` previews, approvals before apply.
- Ensure drift detection runs periodically.

---

99. Scenario:

Terraform provider has a **breaking change** in a new release. How do you mitigate?
Answer:

- Pin provider version in `required_providers`.
- Test new version in staging before prod.
- Use upgrade guides from provider docs.

---

100. Scenario:

Your Terraform apply succeeded, but some resources are **not behaving as expected**. How do you debug?

Answer:

- Use `terraform state show` to inspect resource attributes.
- Check provider debug logs (`TF_LOG=DEBUG`).
- Compare actual cloud console vs state.
- Correct drift or misconfig with updated config.